



Introdução à Herança



Herança em Java

- Para herdar de uma classe, use a palavra reservada `extends`
- A classe herdada é conhecida como classe herdada ou classe filha. A classe da qual ele herda é conhecida como classe base ou classe mãe.
- Quando uma classe A herda de uma classe B, a classe A adquire todos os atributos e comportamento (atributos e métodos) da classe B (embora você não consiga acessar alguns deles por causa dos modificadores de acessibilidade)
- Em Java, você só pode herdar de uma classe (sem contar a classe `Object` da qual todas as classes herdam implicitamente). As interfaces Java foram concebidas para preencher parcialmente essa lacuna.
- Não há limite para o número de classes que podem herdar de uma determinada classe
- Não há limite para o nível de profundidade na árvore de herança entre as classes
- É possível evitar que uma classe seja herdada usando o modificador `final` ao declará-la
- Quando uma classe `Car` herda de uma classe `Vehicle`, um objeto da classe `Car` agora pode ser considerado do tipo `Vehicle`, mas não ao contrário.
- Quando uma classe herda de outra, ela é forçada a implementar construtores apropriados para invocar aqueles da classe base



Modificadores em Java

Os modificadores de acessibilidade permitem definir o nível de acesso que outras classes terão nos atributos ou métodos da classe que os define. A própria classe sempre será capaz de acessar seus atributos / métodos, independentemente do modificador usado

- “Por omissão” (“package”): Somente classes de um mesmo pacote podem acessar os atributos / métodos assim definidos.
- public: Qualquer classe pode acessar os atributos / métodos assim definidos
- protected: Somente classes que herdam ou pertencem ao mesmo pacote podem acessar os atributos / métodos assim definidos.
- privado: Apenas as classes herdadas podem acessar os atributos / métodos assim definidos



Classes abstratas

- Classes abstratas são classes definidas com o único propósito de herdar de outras classes.
- Eles são definidos com a palavra reservada `abstract` -> `public abstract class Vehicle { ... }`
- Não é possível instanciar objetos de uma classe definida como abstrata
- Classes abstratas podem definir métodos sem corpo. Isso significa que será obrigatório que quem herde desta classe terá tem que defini-lo (a menos que também seja abstrato). -> `public abstract void doSomething ();`
- O fato de haver métodos abstratos significa que diferentes objetos que herdam da mesma classe terão os mesmos métodos mas implementação diferente. Quando estes são tratados por meio de variáveis / parâmetros da classe base, veremos que o mesmo método fará coisas diferentes com base no objeto real para o qual essa variável aponta -> Polimorfismo



Sobrescrever métodos

- Qualquer classe que herde de uma classe base poderá sobrescrever os métodos que herda.
 - Para estendê-lo, adicionando mais código (é possível invocar a versão herdada e adicionar novo código)
 - Para substituí-lo por outra implementação (simplesmente redefinindo o método novamente)
- Quando um método sobrescrever outro já herdado, ele deve ser anotado com @Override
- Se o método simplesmente sobrecarrega o método herdado (assinatura diferente) não está substituindo o herdado, é outro método

Exercício 1



Oficina

O proprietário de uma oficina contratou você para desenvolver um aplicativo para gerenciar seu trabalho. Você se encontra em sintonia que deve projetar e implementar as classes necessárias para desenvolver a referida aplicação. Na oficina conserta-se Carros, Motos e vários trabalhos mecânicos. Dos carros, eles desejam armazenar o número do chassi, placa, modelo, cor, quilometragem e capacidade do porta-malas. Das motocicletas, eles também querem salvar o número do chassi, placa, modelo, cor, quilometragem e também querem saber se uma motocicleta tem uma carenagem e seu peso. Em ambos os casos, para motocicletas e carros você vai querer armazenar se eles foram reparados ou não. Além disso, os mecânicos querem armazenar o nome, sobrenome, e-mail, especialidade (Carro ou Moto) e o número de reparos que eles fizeram desde que trabalharam lá. Além disso, o veículo sempre será salvo por cada mecânico que está trabalhando.



- Em relação às operações de carro, o seguinte é esperado:
 - Uma operação que permite adicionar quilômetros ao carro
 - Uma operação para pintar o carro em outra cor
- Essas operações serão necessárias para motocicletas:
 - Uma operação para retirar a carenagem da motocicleta (lembre-se que o peso diminuirá em 2 kg)
 - Uma operação para pintar a motocicleta em outra cor
- No caso da mecânica, as operações serão as seguintes
 - Uma operação que permite que qualquer veículo seja reparado, o que marcará o veículo como reparado
 - Uma operação que permite sinalizar que um veículo entra na oficina, marcando-o como não reparado, para que haja evidências de que é necessário trabalhar em ele
 - De alguma forma teremos que conseguir atribuir um veículo ao mecânico, para que quando ele o consertar, não seja mais modificado.

Implementar as classes necessárias com seus atributos, construtores, getters e setters, e as operações indicadas



Civilização

Agora queremos projetar as classes para um videogame de estratégia no estilo Civilization. Nele, os usuários poderão criar seus próprios personagens e construir uma cidade que evoluirá para conquistar outros jogadores. Cada jogador quer armazenar seu apelido, nome, sobrenome, e-mail e tempo de jogo (em segundos). Você deseja armazenar o nome de cada personagem, época a que pertence, o nível e o dinheiro obtido até agora. Existe uma operação que permite adicionar quilômetros ao carro lembre-se que cada usuário pode ter um personagem e deve ser cadastrado qual é. Como cada personagem pode criar uma cidade (e deve ser assim registrada) o nome, nível, número de habitantes e tipo de cidade (aldeia, vila ou cidade).

- Espera-se que os jogadores sejam capazes de realizar as seguintes operações:
 - **Uma operação para adicionar tempo de jogo (será executada toda vez que um jogo terminar para aumentar o tempo gasto)**
- Espera-se que os personagens sejam capazes de realizar as seguintes operações:
 - **Uma operação que permite criar uma cidade indicando seu nome. O nível inicial e o número de habitantes serão 0, e o tipo de cidade será Vila. A cidade deve estar ligada ao personagem.**
- Para cidades, as seguintes operações serão necessárias
 - **Uma operação que permite elevar o nível da aldeia. Nesta operação, deve-se levar em consideração que quando o nível ultrapassa o valor 10 a cidade mudará para o tipo vila e quando exceder o valor 20 mudará para o tipo de cidade.**
- Para todos os casos, implemente as classes, atributos, construtores necessários e as operações solicitadas.



TokioCalendar

Projetar as classes de um aplicativo que funcione como um calendário, onde os diferentes usuários do mesmo possam gerenciar seus tempo e usá-lo. A ideia é que a aplicação tenha seus próprios sistemas de usuários, que armazenará me de nome de usuário, senha, e-mail, telefone celular, endereço e tipo de usuário (que pode ser administrador, usuário ou convidado).

- A ideia principal da aplicação é poder anotar tarefas ou eventos pendentes para que sejam registrados e possam ser consultados posteriormente. Além disso, a aplicação avisará (se o usuário assim o indicar) quando, por exemplo, aproxima a data de um determinado evento.
- É feita uma distinção entre tarefas e eventos:
 - **Uma tarefa é algo que um usuário deve fazer. Não tem data atribuída, apenas um nome e uma descrição**
 - **Um evento é algo que ocorrerá em uma determinada data. Para cada um, armazenaremos o nome, a descrição e o local. Você pode configurar para que o aplicativo avise o usuário de diferentes maneiras (por tela, por SMS ou por e-mail eletrônico) e com a antecedência que o usuário indica.**
- Além disso, os usuários podem ter seus contatos cadastrados (com nome, sobrenome, e-mail e telefone), que podem ou não ser usuários do aplicativo, para que possam compartilhar tarefas e / ou eventos com eles. Assim, será interessante saber quais contatos estão vinculados a um determinado evento e / ou tarefa porque também serão notificados quando o usuário o for.



Dúvidas?

Tokio.